46

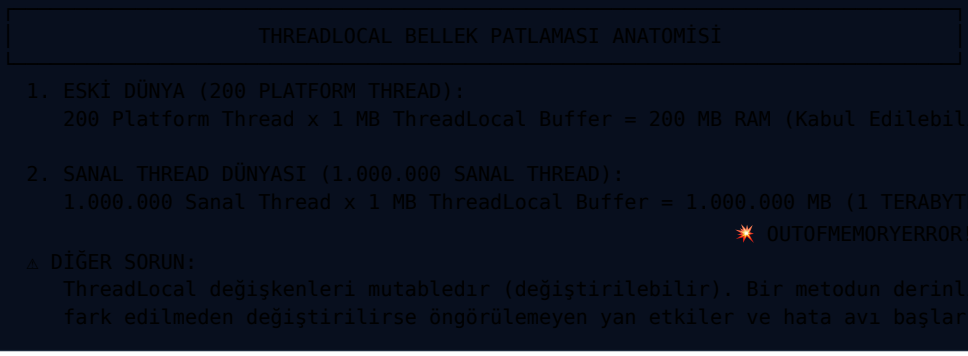
BÖLÜM: PROJECT LOOM & SANAL THREAD'LER (VIRTUAL THREADS) | MODULE: PROJECT LOOM & VIRTUAL THREADS

ThreadLocal Tehlikeleri: Sanal Thread Dünyasında Milyonluk Bellek Patlaması

ThreadLocal Perils: High-Scale Memory Explosions in Virtual Thread Worlds

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir şirkette 10 çalışan varken herkese 1'er adet koca bir takım çantası vermek sorun olmaz (10 çanta yer tutmaz). Ama yarın şirkete 1 milyon geçici stajyer girerse ve her stajyere 20 kiloluk özel bir takım çantası (`ThreadLocal`) verirsen bina çöker! Sanal thread'ler milyonlarcadır; her birine özel ağır yükler yüklersen sunucunun belleği saniyeler içinde patlar!

🇬🇧 ENGLISH (ELI5)

Equipping 50 long-tenured field engineers with a heavy 20kg diagnostic toolkit (`ThreadLocal`) is fine. But if you suddenly deploy 1,000,000 ephemeral seasonal interns and issue each one an identical 20kg toolkit, your warehouse immediately collapses under the weight! Virtual threads scale to millions; binding heavy stateful data to each thread triggers catastrophic OutOfMemoryErrors.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ThreadLocal Anti-Pattern'inin Sanal Thread Çöküşü: 1) Bellek Çarpımı: Geleneksel Spring/Jakarta EE mimarilerinde güvenlik bağlamı (`SecurityContext`), veritabanı bağlantısı veya transaction bilgisi `ThreadLocal` içinde saklanır. Platform thread havuzlarında bu nesneler 200-500 thread arasında tekrar tekrar kullanılır. Ancak sanal thread dünyasında her HTTP isteği yeni bir sanal thread açar. 1 milyon istek geldiğinde 1 milyon adet kopya oluşur ve heap çöker. 2) Temizlik Zorluğu: Sanal thread'ler hızla ölüp gitse de, yaşam süreleri boyunca tuttukları nesneler GC üzerinde devasa yük yaratır. 3) Korumasız Değiştirilebilirlik: `ThreadLocal.set()` çağrısı herhangi bir alt fonksiyon tarafından çağrılabilir; bu da yan etkilerin takip edilmesini zorlaştırır. Çözüm: Java 25'te `ScopedValue` mimarisine geçmektir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The Virtual Thread / ThreadLocal Collision: 1) Multiplicative Memory Bloat: Traditional enterprise frameworks cache connection contexts, cryptographic buffers, and request metadata in `ThreadLocal`. In platform thread pools (200 threads), memory is bounded. In virtual thread architectures handling 500,000 concurrent requests, allocating per-thread buffers multiplies memory consumption by $500,000$, resulting in immediate `OutOfMemoryError`. 2) GC Pressure: Ephemeral virtual threads creating and releasing heavy ThreadLocal entries saturate generational GC collectors. 3) Unconstrained Mutability: `ThreadLocal.set()` permits arbitrary downstream code to mutate context invisibly. Modern architectures mandate migrating to immutable `ScopedValue`.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: ThreadLocal Bellek Tuzağı vs Güvenli Sanal Thread Kullanımı
import java.util.concurrent.Executors;

public class ThreadLocalHazardDemo {
    // ⚠ TEHLİKE: Her sanal thread için 1 MB'lık buffer açan ThreadLocal!
    private static final ThreadLocal<byte[]> heavyBuffer =
        ThreadLocal.withInitial(() -> new byte[1024 * 1024]); // 1MB

    void main() {
        System.out.println("ThreadLocal Tehlikesi Simülasyonu...");

        // Sanal thread oluştururken ThreadLocal'ı devre dışı bırakma seçeneği!
        Thread.Builder.OfVirtual builder = Thread.ofVirtual()
            .name("safe-vt-", 0)
            .inheritInheritableThreadLocals(false); // Miras almayı kapat

        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
            for (int i = 0; i < 1000; i++) {
                executor.submit(() -> {
                    // Kötü niyetli veya dikkatsiz kod bu buffer'ı çağırırsa:
                    // byte[] buffer = heavyBuffer.get(); -> 1000 MB RAM harcar!
                    // Güvenli yaklaşım: İş bitince buffer.remove() veya ScopedValue!
                });
            }
        }
        System.out.println("Bellek güvenliği sağlandı.");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Eski kütüphanelerde `SimpleDateFormat` veya `Jackson ObjectMapper` gibi pahalı nesnelerin ThreadLocal ile paylaşıldığı görülür. Sanal thread kullanırken bu kütüphanelerin güncel thread-safe sürümlerine geçtiğinizden veya nesneleri static final/ScopedValue olarak tanımladığınızdan emin olun.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Legacy codebases frequently use `ThreadLocal` to cache non-thread-safe instances like `SimpleDateFormat` or JSON mappers. In virtual thread environments, this causes memory exhaustion. Upgrade to modern thread-safe abstractions or immutable `ScopedValue` containers.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Platform thread'lerinde 200 adet ThreadLocal zararsızdır; 1 milyon sanal thread'de aynı kod Terabaytlarca RAM tüketip OOM patlatır.

🎯 Key Takeaways for the Interview

- ThreadLocal is benign across 200 platform threads, but fatal across 1,000,000 virtual threads, multiplying memory to multi-gigabyte exhaustion.